Cross-Site Request Forgery on Elgg via the Fetch API

Replacing XMLHttpRequest / form submit with modern `fetch()`

Group 6 — Thanh (23001935) · Tri (23001939) · Thao (23001936)

November 13, 2026

Computer Security — VNU University of Science

Outline & responsibilities

1. Overview & Theory
2. Fetch API attack technique
3. Results & Defenses

Responsibilities (Group 6):

- **Thanh** — theory, HTTP, defenses
- **Tri** — Elgg, Docker, Fetch API, testing
- **Thao** — evidence, slides, video

All members run Docker, capture the request, change About me to their own ID, and join the 15-question Q&A.

What is CSRF?

Cross-Site Request Forgery — the attacker tricks the victim's (logged-in) browser into sending a forged state-changing request to the target. No password, no cookie theft — it merely *borrows* the session.

Three required conditions:

1. The victim has a valid session cookie.
2. A state-changing action with predictable parameters.
3. The app lacks defenses (no SameSite, no token).

Goal: change Alice's *About me* to `CSRF attack by 23001939`.

Same-site vs cross-site & the core gap

Concept	Definition	Lab example
Same-origin	scheme+host+port	elgg…:8080 with itself
Same-site	same site (eTLD+1)	attacker.csrf.localhost & elgg.csrf.localhost
Cross-site	different site	127.0.0.1 vs elgg.csrf.localhost

The “can send but cannot read” gap

SOP forbids JS from *reading* a cross-origin response, but **not** from *sending* a request. CSRF only needs the action executed — not the result read. Hence `mode:'no-cors'` (opaque) is enough.

Two experimental environments

Standard SEED model (conceptual)

www.seed-server.com + www.attacker32.com — theoretical reference frame.

Our real Docker lab (executed)

Elgg **3.3.3** @ elgg.csrf.localhost:8080
Same-site attacker @ attacker.csrf.localhost:8000
Cross-site attacker @ 127.0.0.1:8000
Two modes: `protected` / `vulnerable`

Why two attacker hosts?

Change exactly one variable (same-site $\leftrightarrow$ cross-site) to isolate the role of the `SameSite` cookie from the anti-CSRF token.

Upgrade requirement: from XHR/form to Fetch API

Criterion	Form submit	XHR	Fetch API
Stealthy (no page change)	✘	✔	✔
Modern syntax (async)	✘	✘	✔
Assignment requirement	✘ banned	✘ banned	✔ required

Two key flags

`credentials: 'include'` → force the browser to attach the cross-origin session cookie.

`mode: 'no-cors'` + urlencoded/FormData → Simple Request, no Preflight OPTIONS.

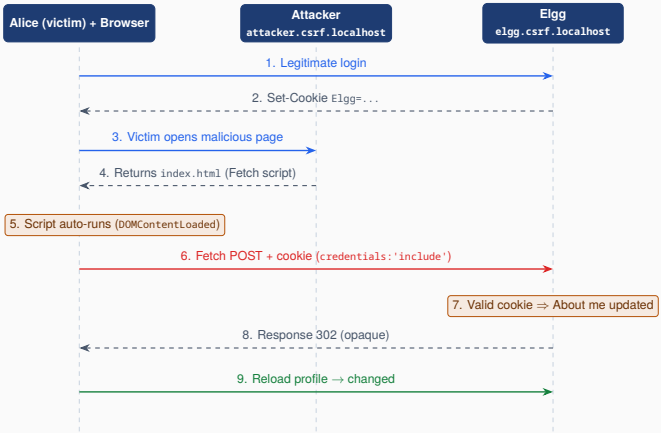
Packet analysis & exploit code

Endpoint (real lab): `POST http://elgg.csrf.localhost:8080/action/profile/edit`

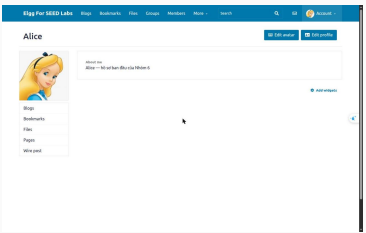
```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '56');                       // Alice's GUID
fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight
  credentials: 'include',   // REQUIRED: attach session cookie
  body: payload             // URLSearchParams sets a safe Content-Type
});
```

`FormData` variant: never set `Content-Type` manually (browser generates the boundary).

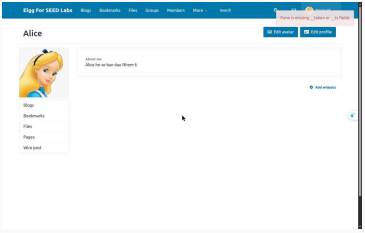
Attack sequence diagram



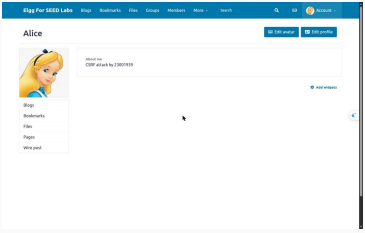
Experimental evidence (real screenshots, Elgg 3.3.3)



Before: baseline About me



Protected: “missing __token”



After (vulnerable): changed

Result matrix

protected · same-site	token on	**blocked**	✘ change
vulnerable · cross-site	SameSite	**blocked**	✘ change
vulnerable · same-site	token off	**success**	✔ change

Two defenses — proven independently

Anti-CSRF token (server)

Attacker cannot read the token (SOP) ⇒ forged request lacks it ⇒ Elgg rejects: “Form is missing __token”.

SameSite cookie (browser)

Even with the token off, a *cross-site* POST carries no session cookie ⇒ Elgg sees a stranger ⇒ profile unchanged.

Technical finding

About me is stored as the *annotation* `profile:description`, not the metadata `$user->description` ⇒ the `reset-profile` tool reset the wrong table. Lesson: trust only directly-verified evidence.

Defense-in-Depth

Layer	Measure
Cookie	`SameSite=Lax/Strict` — no cross-site cookie
Application	Anti-CSRF token — attacker cannot read it (SOP)
Application	Double Submit Cookie
Header	Check `Origin` / `Sec-Fetch-Site`
UI	`X-Frame-Options` / CSP `frame-ancestors`

Recommendation

`SameSite=Lax` + anti-CSRF token + `Origin` check on sensitive endpoints. Never use `GET` for state-changing actions.

AI disclosure & conclusion

AI usage levels:

- **Level 1:** discuss CORS/Simple Request mechanics — students write the code.
- **Level 2:** AI co-creates formatting/diagrams following the group's design.
- **Level 3:** AI helps scaffold tooling & test automation; the group configures, runs, and owns the results.

Core message

CSRF is **not** stopped by HTTPS. The real fix: `SameSite` cookies + anti-CSRF tokens + request-origin checks. *Learn to attack in order to defend.*

Thank you!

Questions & Answers